

Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Радіотехнічний факультет
Кафедра прикладної радіоелектроніки

Звіт
про лабораторну роботу №2 за темою
«Шифрування та розшифрування на машині «Енігма»»
з дисципліни
«Захист даних»

Викладач:

Навроцький Денис Олександрович

Виконав:

Студент групи РС-61мн

Гандзюк Дмитро Васильович

Київ – 2026 р.

Лабораторна робота №2

Шифрування та розшифрування на машині «Енігма»

Мета роботи: реалізувати шифр, побудований на двох базових криптографічних примітивах — перестановці (зсуві) та заміні (підстановці), — на прикладі спрощеної моделі шифрувальної машини «Енігма», довести коректність реалізації на офіційних валідаторах навчальної платформи CodinGame, а також самостійно оцінити криптографічну стійкість отриманого шифру: дослідити статистичні властивості шифротексту, визначити фактичний розмір простору ключів і практично реалізувати атаки у чотирьох розглянутих сценаріях — від перебору ключа до відновлення відкритого тексту лише за шифротекстом.

Хід роботи

Постановка задачі

Задача взята з навчальної платформи CodinGame (розділ Training, рівень складності Easy) і має назву «Encryption/Decryption of Enigma Machine». У ній відтворено спрощений принцип роботи шифрувальної машини «Енігма», у якому обидва класичні криптографічні примітиви подані в чистому, нічим не замаскованому вигляді.

Перший примітив — **перестановка алфавіту**, реалізована як шифр Цезаря з інкрементним ключем: перший символ повідомлення зсувається на N позицій, другий — на $N+1$, третій — на $N+2$ і так далі. Другий примітив — **заміна**: отриманий після зсуву текст послідовно пропускається крізь три ротори, кожен з яких є довільною підстановкою (бієкцією) 26-літерного латинського алфавіту.

Термінологічне уточнення. У постановці задачі зсув розглядається як перестановка алфавіту. У строгій класифікації шифрів він є підстановкою символів, оскільки їхні позиції в повідомленні не змінюються; перестановний шифр, навпаки, переставляє символи самого повідомлення (Handbook of Applied Cryptography, § 1.5.2). Далі в роботі збережено термінологію умови задачі.

Формат вхідних даних задачі:

рядок 1 ENCODE | DECODE
 рядок 2 початковий зсув N, $0 \leq N < 26$
 рядки 3–5 три ротори по 26 великих латинських літер
 рядок 6 повідомлення, $1 \leq \text{довжина} < 50$, лише A..Z

Результатом роботи програми має бути одне слово — зашифроване або розшифроване повідомлення. Наведений в умові приклад зручно розібрати покроково: для повідомлення AAA та початкового зсуву $N = 4$ зсув дає EFG ($A+4$, $A+5$, $A+6$), перший ротор перетворює це на JLC, другий — на BHD, третій — на KQF. Саме KQF і є очікуваною відповіддю.

Математична модель шифру

Позначимо літери числами від 0 до 25 ($A = 0$, $Z = 25$), а всі операції виконуватимемо в кільці лишків за модулем 26. Нехай R_1 , R_2 , R_3 — підстановки, задані роторами, N — початковий зсув, m — відкритий текст, c — шифротекст. Тоді пряме та зворотне перетворення записуються так:

$$c[i] = S[(m[i] + N + i) \bmod 26], \quad S = R_3 \cdot R_2 \cdot R_1$$

$$m[i] = (S^{-1}[c[i]] - N - i) \bmod 26$$

З цього запису випливають три спостереження, на яких надалі тримається вся дослідницька частина роботи.

Спостереження 1. Три ротори — це один ротор. Композиція бієкцій є знову бієкцією, тому добуток $S = R_3 \cdot R_2 \cdot R_1$ — це одна підстановка алфавіту. Каскад із трьох роторів не дає нічого такого, чого не дав би один належно підібраний ротор. Це твердження перевірено не міркуванням, а програмно: метод `equivalent_single_rotor()` будує машину з одним ротором і нульовим зсувом, а модульний тест порівнює шифротексти обох машин на 50 випадкових конфігураціях — вони збігаються символ у символ.

Спостереження 2. Шифр є поліалфавітним із періодом рівно 26. На позиції i застосовується алфавіт заміни з номером $p = (N + i) \bmod 26$, причому всі 26 алфавітів породжені однією підстановкою S за правилом $A_p(m) = S[(m + p) \bmod 26]$. Отже, послідовність алфавітів циклічно повторюється з періодом 26 незалежно від вмісту повідомлення.

Спостереження 3. Увесь ключ стискається до 26 чисел. Введемо поняття *ефективного ключа* D , визначивши $D[x] = (S^{-1}[x] - N) \bmod 26$. Тоді розшифрування набуває вигляду $m[i] = (D[c[i]] - i) \bmod 26$. Це повний опис шифру: початковий зсув N «втоплено» всередину підстановки і окремо його знати не потрібно. Саме ефективний ключ D відновлюють усі чотири

реалізовані далі атаки.

Програмна реалізація

Розв'язок виконано мовою Python 3 (основна реалізація) і продубльовано мовою C# на платформі .NET 8 (незалежна реалізація для перехресної перевірки). Вихідний код самодостатнього розв'язку для платформи наведено в додатку А. Решта вихідних текстів — ядро шифру, модуль криптоаналізу, статистичні метрики, набір автоматичних тестів і реалізація мовою C# — доступні в репозиторії роботи, посилання на який наведено в списку використаних джерел.

Головне архітектурне рішення впливає зі спостереження 1: каскад роторів згортається в одну таблицю підстановки до початку обробки повідомлення. Завдяки цьому у внутрішньому циклі на кожен символ припадає лише одне звертання до таблиці. У повній версії бібліотеки попередньо обчислюються всі 26 алфавітів заміни (матриця 26×26 , тобто 676 байт), тому внутрішній цикл узагалі не містить арифметичних операцій — тільки індексацію.

Обчислювальна складність шифрування становить $O(L + 26k)$, де L — довжина повідомлення, а k — кількість роторів; додаткова пам'ять — $O(26^2)$ і не залежить від довжини повідомлення. Розшифрування симетричне та має ту саму складність. Ядро шифру й усі атаки написані виключно на стандартній бібліотеці Python, без зовнішніх залежностей.

Верифікація розв'язку

Правильність реалізації підтверджено на чотирьох незалежних рівнях: офіційними валідаторами платформи, збереженими в репозиторії офіційними тестами, набором властивісних тестів і перехресним порівнянням двох реалізацій різними мовами.

Розв'язок надіслано на перевірку платформі CodinGame. Усі шість видимих тестів пройдено, і, що важливіше, пройдено приховані валідатори: підсумкова оцінка становить **100 %**. Надісланий код у вікні редактора платформи наведено на рисунку 2.1, результат прогону всіх шести тестів — на рисунку 2.2, підсумкову оцінку валідаторів — на рисунку 2.3.

```

1
2 import sys
3
4 A = ord("A")
5 M = 26
6
7
8 def main() -> None:
9     data = sys.stdin.read().split()
10    mode = data[0].upper()
11    shift = int(data[1]) % M
12    rotors = data[2:5]
13    message = data[5] if len(data) > 5 else ""
14
15    # Collapse the rotor cascade into one substitution table.
16    s = list(range(M))
17    for wiring in rotors:
18        table = [ord(ch) - A for ch in wiring]
19        s = [table[x] for x in s]
20
21    if mode == "ENCODE":
22        out = [
23            chr(A + s[(ord(ch) - A + shift + i) % M])
24            for i, ch in enumerate(message)
25        ]
26    else:
27        inv = [0] * M
28        for i, v in enumerate(s):
29            inv[v] = i
30        out = [
31            chr(A + (inv[ord(ch) - A] - shift - i) % M)
32            for i, ch in enumerate(message)
33        ]
34
35    print("".join(out))
36
37
38 if __name__ == "__main__":
39     main()
40

```

Рисунок 2.1 – Надісланий розв’язок у вікні редактора CodinGame

Test cases			
01	Encode 3		▶ PLAY TESTCASE
02	Encode 23		▶ PLAY TESTCASE
03	Decode 21		▶ PLAY TESTCASE
04	Encode 21		▶ PLAY TESTCASE
05	Encode 42		▶ PLAY TESTCASE
06	Decode 49		▶ PLAY TESTCASE

Рисунок 2.2 – Усі шість тестів платформи позначені як пройдені

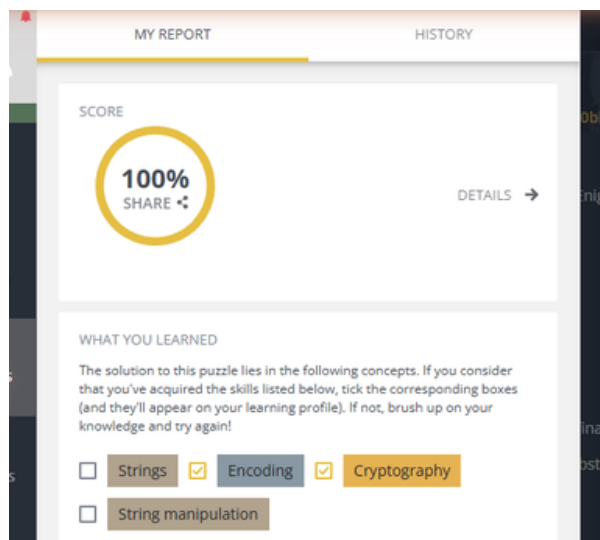


Рисунок 2.3 – Звіт платформи: оцінка валідаторів — 100 %

Входи та еталонні виходи всіх шести офіційних тестів збережено в репозиторії у файлі `tests/official_cases.json`, тому їх можна прогнати локально без доступу до платформи. Перелік тестів наведено в таблиці 2.1.

Таблиця 2.1 – Офіційні тести CodinGame

№	Назва	Режим	N	Довжина	Результат
1	Encode 3	ENCODE	4	3	пройдено
2	Encode 23	ENCODE	7	23	пройдено
3	Decode 21	DECODE	9	21	пройдено
4	Encode 21	ENCODE	9	21	пройдено
5	Encode 42	ENCODE	9	42	пройдено
6	Decode 49	DECODE	5	49	пройдено

Результат локального прогону цих тестів:

```
$ python run.py selftest
Офіційні тести CodinGame
```

```
-----
[OK] Encode 3      ENCODE -> KQF
[OK] Encode 23     ENCODE -> ALWAURKQEQQLRAWZHUYKVN
[OK] Decode 21     DECODE -> EVERYONEISWELCOMHERE
[OK] Encode 21     ENCODE -> PQSACVVTOISXFXCIAMQEM
[OK] Encode 42     ENCODE -> PQSACVVTOISXFXCIAMQEMDZIXFJJSTQIENEFQXVZYV
[OK] Decode 49     DECODE -> THEQUICKBROWNFOXJUMPSOVERALAZYSphinxOFBLACKQUARTZ
-----
```

Пройдено 6 з 6

Окрім офіційних тестів, написано набір із 86 автоматичних тестів, що перевіряють: оборотність перетворення на 300 випадкових конфігураціях; періодичність шифру (перші 26 символів шифротексту рядка з 52 літер А збігаються з наступними 26); еквівалентність каскаду з трьох роторів одному ротору; модульність зсуву ($N = 3$ і $N = 29$ дають однаковий результат); некомутативність порядку роторів; відхилення некоректних роторів і символів поза алфавітом А..Z. Реалізація мовою С# проходить ті самі шість офіційних тестів і 2000 перевірок оборотності, а окремий тест порівнює вихід обох реалізацій символ у символ. Увесь набір виконується автоматично при кожній зміні коду засобами GitHub Actions на трьох версіях Python.

Під час підготовки роботи в самому тестовому наборі було виявлено й усунуто помилку: вивід програми на С# читався в кодуванні консолі Windows замість UTF-8, через що порівняння з українським рядком не спрацьовувало. Цей випадок підтверджує необхідність фактичного виконання всіх перевірок.

Частотний аналіз шифротексту

Перше, що перевіряє криптоаналітик, — чи зберігає шифротекст частотну структуру мови. Для експерименту взято англomовний текст обсягом 2035 літер і зашифровано його з початковим зсувом $N = 7$. Порівняння частот літер відкритого тексту, шифротексту й еталонних частот англійської мови наведено на рисунку 2.4.

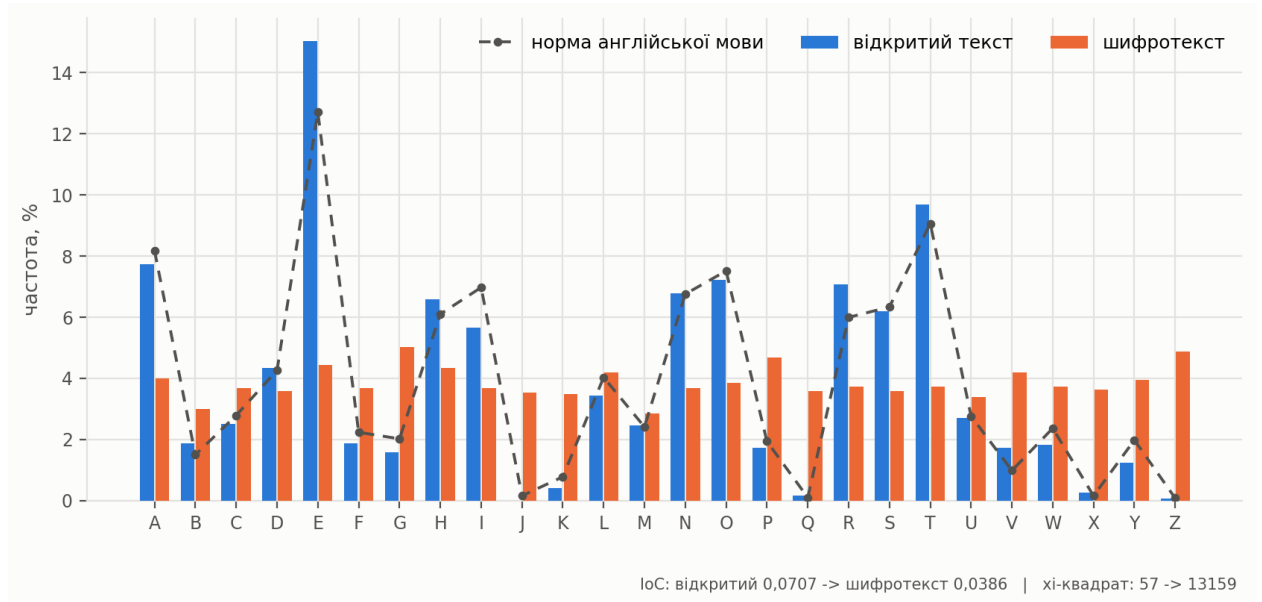


Рисунок 2.4 – Інкрементний зсув вирівнює частоти літер шифротексту

Результат на перший погляд свідчить на користь шифру. Інкрементний зсув «розмазує» кожен літеру відкритого тексту по всіх 26 позиціях алфавіту, і розподіл літер шифротексту стає майже рівномірним: максимальне відхилення частоти будь-якої літери від рівномірних 3,85 % становить лише 1,17 відсоткового пункту. Кількісні показники зведено в таблиці 2.2.

Таблиця 2.2 – Статистичні характеристики тексту до та після шифрування

Показник	Відкритий текст	Шифротекст	Орієнтир
Індекс відповідності (ІоС)	0,0707	0,0386	англ. 0,0667 / випадк. 0,0385
Хі-квадрат до частот англійської мови	57	13159	менше — ближче до мови
Емпірична ентропія окремих символів, біт	4,105	4,689	максимум 4,700

Індекс відповідності впав з 0,0707 (типове значення для англійської мови) до 0,0386, що практично збігається зі значенням для рівномірно

випадкової послідовності. Емпірична ентропія окремих символів зростає до 4,689 біта при теоретичному максимумі 4,700. Отже, пряме зіставлення загальної гістограми з частотами англійської мови для цього шифротексту малоефективне. Це, однак, не означає статистичної невідрізненості всієї послідовності від випадкової: наведені показники не виключають позиційних залежностей, які й виявляє наступний розділ.

Саме на цьому місці й виникає найпоширеніша помилка — висновок про стійкість шифру. Наступні розділи показують, що для дослідженої конструкції він не обґрунтований.

Визначення періоду шифру

Ключ до злому дає індекс відповідності, застосований не до всього тексту, а до його частин. Розіб'ємо шифротекст на p стовпців за індексом $i \bmod p$. Якщо припущений період p збігається зі справжнім, то кожен стовпець виявиться результатом однієї й тієї самої моноалфавітної заміни, а індекс відповідності всередині стовпця повернеться до значення, характерного для природної мови (ІоС інваріантний щодо простої заміни). Профіль залежності середнього ІоС стовпця від припущеного періоду наведено на рисунку 2.5.

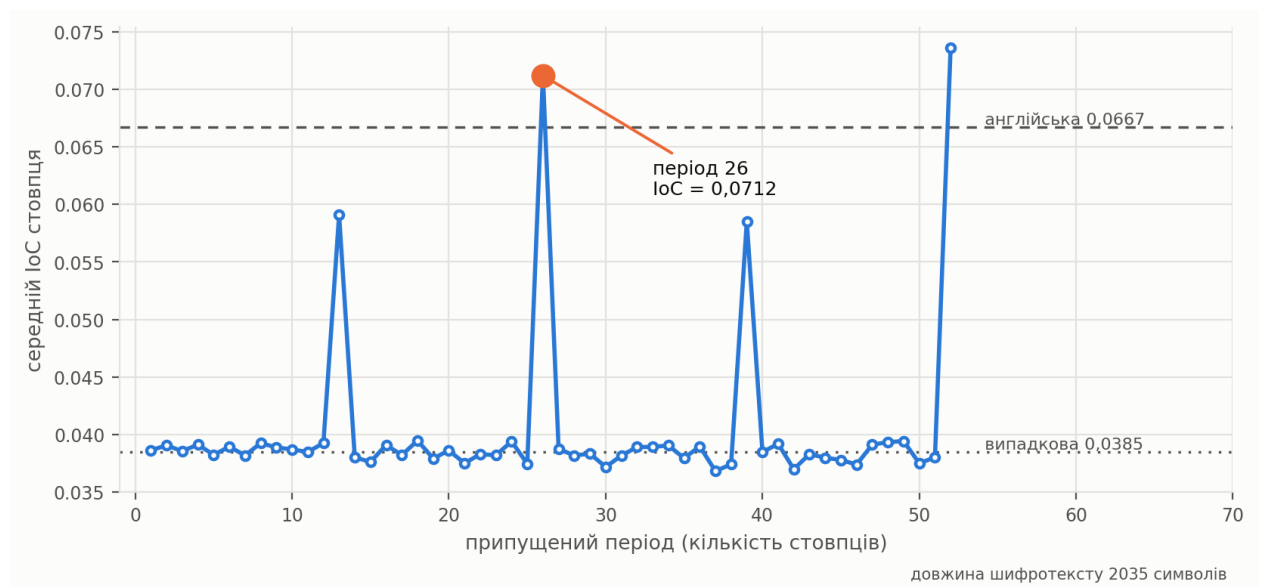


Рисунок 2.5 – Профіль індексу відповідності викриває поліалфавітний період 26

Найвищі значення спостерігаються для $p = 26$ і $p = 52$ (ІоС = 0,0712 проти 0,0386 для тексту в цілому), помітно підвищені — для $p = 13$ і $p = 39$. Пояснення дає кількість різних алфавітів, що потрапляють в один стовпець:

вона дорівнює $26 / \text{НСД}(p, 26)$. Для $p = 26$ і 52 це один алфавіт, тобто стовпець є чистою моноалфавітною заміною; для $p = 13$ і 39 у стовпці чергуються два алфавіти, що теж підвищує ІоС відносно всієї вибірки, але слабше. Зауважимо, що з цих значень кратним 26 є лише $52: 13$ — дільник 26 , а 39 кратне 13 . Отже, період визначено без будь-якого знання про ротори чи початковий зсув.

Ієрархія атак на шифр

Реалізовано чотири атаки, впорядковані за спаданням припущень про можливості зломисника — від найсильніших до найслабших. Усі вони відновлюють один і той самий об'єкт — ефективний ключ D . Для оцінювання правдоподібності кандидатів використано статистичну модель англійської мови на квадриграмах, натреновану на корпусі обсягом $5\,623\,458$ літер текстів суспільного надбання (у репозиторії зберігається лише похідна статистика частот, а не самі тексти). Зведені результати наведено в таблиці 2.3.

Таблиця 2.3 – Результати атак на повідомленні завдовжки 180 символів

Атака	Що відомо зломиснику	Вартість	Час	Ключ
Перебір зсуву	ротори	26 варіантів	0,007 с	відновлено
Відомий відкритий текст	пара текстів	180 символів	< 0,001 с	відновлено
Підібраний відкритий текст	доступ до оракула	1 запит (26 символів)	< 0,001 с	відновлено
Тільки шифротекст	нічого, крім шифротексту	78 481 оцінка	3,04 с	відновлено

Атака 1 — повний перебір зсуву

Найважливіше зауваження до цієї задачі впливає з принципу Керкгоффса: стійкість шифру має залежати виключно від ключа, а не від секретності алгоритму. У розглянутій задачі ротори надруковані просто в умові, і, що показово, в усіх шести офіційних тестах вони однакові (це зафіксовано окремим тестом). Отже, справжнім секретом лишається єдине число — початковий зсув N .

Це дає лише 26 варіантів ключа, тобто близько 4,7 біта ентропії. Атака полягає в розшифруванні шифротексту всіма 26 ключами та виборі найбільш «англійського» результату. Розподіл оцінок правдоподібності по кандидатах наведено на рисунку 2.6.

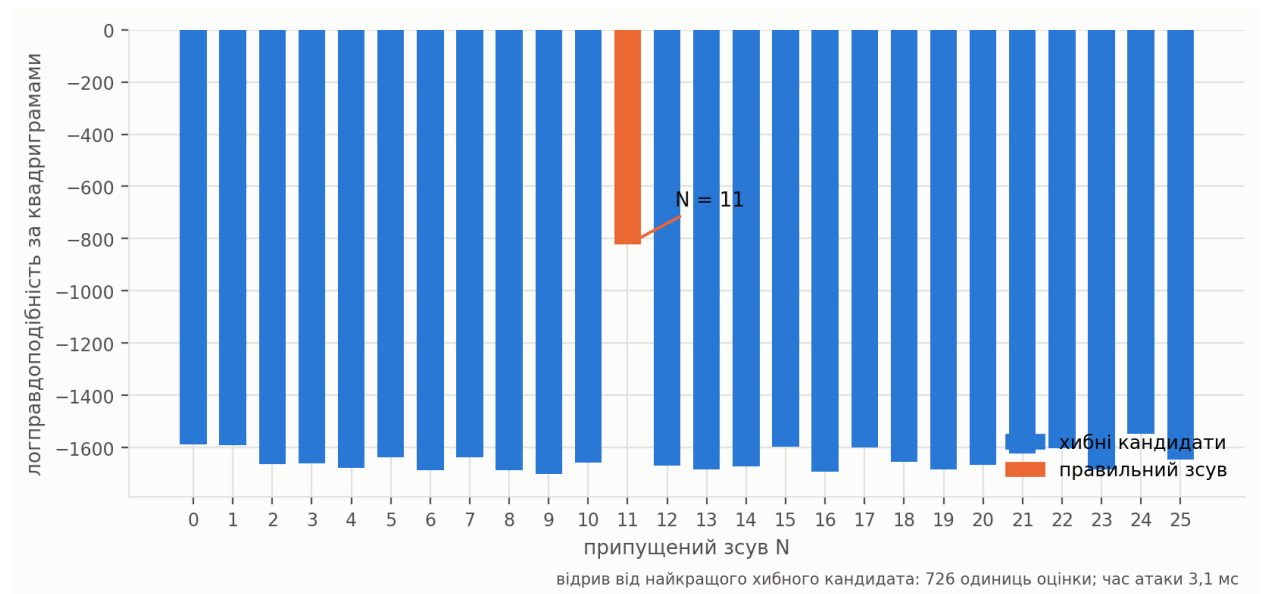


Рисунок 2.6 – Оцінки правдоподібності 26 кандидатів для повідомлення завдовжки 180 символів

Для наведеного повідомлення завдовжки 180 символів перебір завершився за 3,1 мілісекунди, а правильний кандидат відірвався від найкращого хибного на 726 одиниць логарифмічної правдоподібності — вибір однозначний.

Проте з цього не випливає, що атака гарантовано зламує будь-яке повідомлення. Перебір гарантує лише те, що правильне розшифрування *входить* до списку з 26 кандидатів; автоматичний вибір за мовною оцінкою може бути неоднозначним, і для коротких повідомлень це цілком реальна ситуація. Для роторів із умови задачі три різні трилітерні англійські слова дають однаковий шифротекст:

відкритий текст	зсув N	шифротекст
ADD	4	KDW
BEE	3	KDW
ILL	22	KDW

Жодна мовна модель не здатна вибрати між цими варіантами за самим лише шифротекстом. Крім того, у повідомленні, коротшому за чотири символи, немає жодної квадриграми, тож застосована оцінка на них узагалі не спирається. Коректне формулювання таке: перебір зводить задачу до 26

кандидатів, а надійність автоматичного вибору залежить від довжини повідомлення.

Атака 2 — відомий відкритий текст

Якщо зломиснику відомий хоча б фрагмент відкритого тексту разом з відповідним фрагментом шифротексту (так звана «шпаргалка», *crib*), ключ відновлюється взагалі без перебору: кожна відома пара символів дає один елемент ефективного ключа за формулою $D[c[i]] = (m[i] + i) \bmod 26$.

Виникає природне запитання: скільки відомого тексту для цього потрібно? Задача зводиться до класичної задачі про збирання купонів, для якої очікуване покриття описується формулою $26 \cdot (1 - (25/26)^L)$. Експериментальна перевірка (по 200 прогонів на кожну точку) показала збіг із теорією із середнім відхиленням 0,061 елемента ключа — рисунок 2.7 і таблиця 2.4.

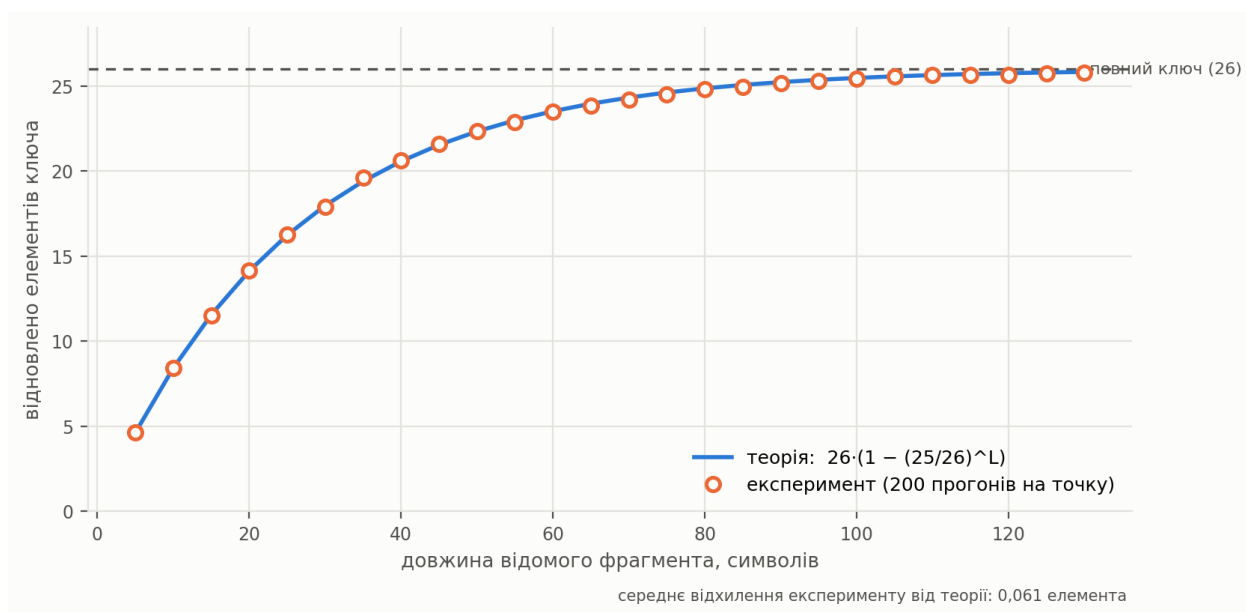


Рисунок 2.7 – Відновлення ключа за відомим відкритим текстом:
експеримент проти теорії

Таблиця 2.4 – Покриття ефективного ключа залежно від довжини відомого фрагмента

Довжина фрагмента L	10	25	50	105
Експеримент	8,44	16,29	22,35	25,56
Теорія	8,44	16,25	22,34	25,58

Щоб зібрати всі 26 елементів ключа, потрібно в середньому близько $26 \cdot H(26) \approx 100$ символів відомого тексту. Проте повний ключ зазвичай і не потрібен: навіть половина елементів дозволяє прочитати більшу частину повідомлення, а решту символів легко відновити за контекстом.

Атака 3 — підбраний відкритий текст

Найдешевша з реалізованих атак. Припустимо, що зловмисник має доступ до шифрувального пристрою як до «чорної скриньки» (оракула) і може подати на вхід довільне повідомлення. Надішлемо рядок із 26 однакових літер А. Тоді $c[i] = S[(0 + N + i) \bmod 26]$, а отже $D[c[i]] = i$ для всіх i від 0 до 25.

Оскільки S — бієкція, усі 26 отриманих символів різні, і повний ефективний ключ відновлюється з **єдиного запиту завдовжки 26 символів**. Ані ротори, ані початковий зсув при цьому знати не потрібно. Коректність атаки перевірено на 25 випадкових конфігураціях машини — у кожному випадку відновлений ключ точно збігся зі справжнім.

Атака 4 — відновлення тексту лише за шифротекстом

Це найслабша модель зловмисника і, відповідно, найцікавіший результат роботи: відомий лише шифротекст, ротори й зсув невідомі повністю. Ключова ідея полягає в тому, що позиційний зсув $-i$ є *публічною структурою, а не секретом*: його не треба вгадувати. Тому перебирати доводиться лише одну бієкцію D , і задача зводиться до звичайної моноалфавітної заміни.

Реалізовано сходження за схилом (hill climbing) у просторі бієкцій з фітнес-функцією на квадриграмах. Початкове наближення будується жадібним частотним методом: для кожної пари «літера шифротексту — кандидат» обчислюється частотна правдоподібність породжених символів, після чого пари розбираються за спаданням вигоди зі збереженням бієктивності. Далі ключ уточнюється випадковими обмінами з кількома незалежними перезапусками. Залежність частки повних зламів від довжини повідомлення наведено на рисунку 2.8 і в таблиці 2.5.

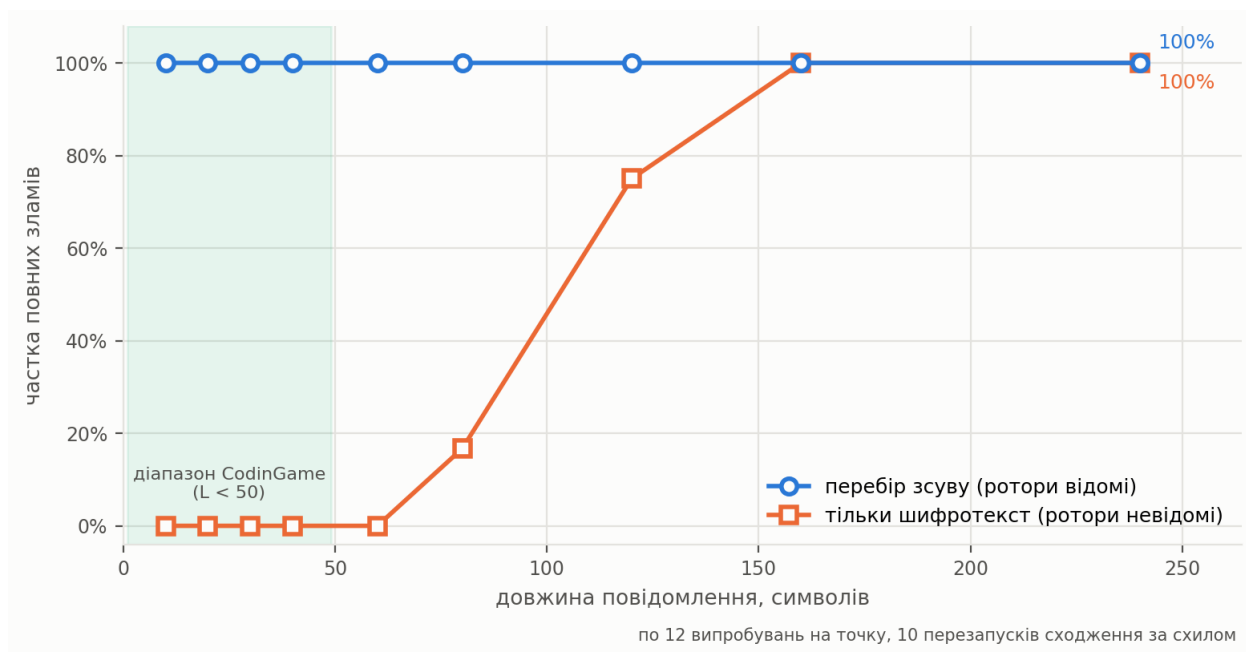


Рисунок 2.8 – Успішність атак залежно від довжини повідомлення

Таблиця 2.5 – Результати серії випробувань атаки лише за шифротекстом

Довжина повідомлення, символів	40	80	120	160	240
Повних відновлень	0 / 12	2 / 12	9 / 12	12 / 12	12 / 12
Частка, %	0,0	16,7	75,0	100,0	100,0
Середній час атаки, с	0,11	0,23	0,35	0,52	0,67

Спостережений поріг варто зіставити з інформаційно-теоретичною оцінкою. За припущення ентропійної швидкості англійського тексту $h \approx 1,5$ біта на символ відстань єдиності для ключа обсягом $26!$ становить $U = H(K) / (\log_2 26 - h) \approx 88,4 / 3,2 \approx 28$ символів. Ця величина характеризує типову інформаційну неоднозначність у прийнятій моделі мови, але не встановлює жорсткої межі для кожного окремого повідомлення й не гарантує успіху конкретного алгоритму. Спостережений поріг (близько 120–160 символів для цієї серії випробувань і вжитих параметрів пошуку) приблизно в 4–6 разів вищий — це типовий розрив між інформаційно-теоретичною оцінкою та можливостями евристичного алгоритму.

Окремо варто зазначити наслідок для умови задачі: платформа обмежує довжину повідомлення умовою $1 \leq L < 50$, тобто 49 символами, — саме той діапазон, у якому атака лише за шифротекстом у цій серії ще не давала надійного результату. Для повідомлень такої довжини залишається атака 1:

ротори в задачі опубліковані, тож перебір 26 значень зсуву триває мілісекунди, хоча для найкоротших повідомлень вибір серед кандидатів може лишатися неоднозначним.

Оцінка простору ключів

Номинально конфігурацію шифру задають три довільні підстановки 26-літерного алфавіту та початковий зсув, тобто $(26!)^3 \cdot 26$ варіантів; за рівноймовірного вибору це відповідає ентропії приблизно 269,8 біта. Проте, як показано у спостереженнях 1 і 3, весь цей набір згортається в одну бієкцію D, тож ентропія ефективного ключа становить $\log_2(26!) \approx 88,4$ біта. Якщо ж урахувати, що ротори опубліковані в умові задачі, залишається 26 варіантів зсуву — приблизно 4,7 біта. Порівняння наведено на рисунку 2.9 і в таблиці 2.6.

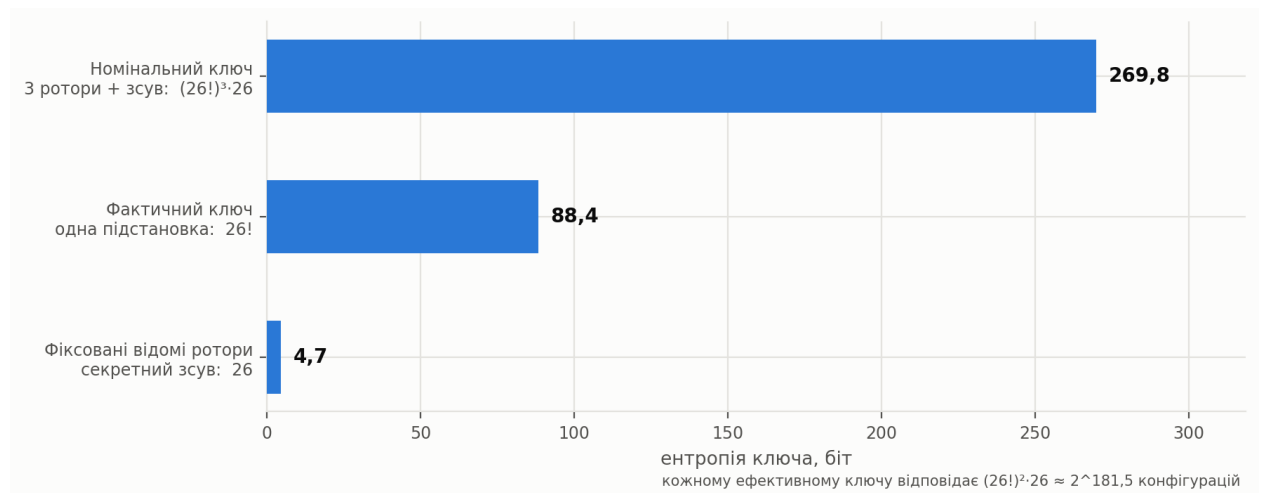


Рисунок 2.9 – Ентропія номінального простору конфігурацій і простору ефективних ключів

Таблиця 2.6 – Простір конфігурацій і простір ефективних ключів

Модель	Кількість варіантів	Ентропія, біт
Номинально: 3 ротори та зсув	$(26!)^3 \cdot 26$	269,8
Фактично: одна підстановка D	$26!$	88,4
Фіксовані відомі ротори, секретний зсув	26	4,7

Отже, з номінальних 269,8 біта на ефективний ключ припадає лише 88,4. Кожному ефективному ключу відповідає $(26!)^2 \cdot 26$, тобто приблизно $2^{181,5}$, різних наборів «ротори + зсув», які задають тотожне перетворення. Це впливає з властивості композиції бієкцій і додатково перевірено

конструктивно: функція `find_key_collision()` будує два різні набори, шифротексти яких збігаються символ у символ на 120 випадкових літерах.

Поширення зміни одного символу

Оцінимо міжсимвольну дифузію: змінюватимемо один символ відкритого тексту й підраховуватимемо, скільки символів шифротексту через це зміниться (по 120 випробувань на кожну точку). Результати наведено на рисунку 2.10 і в таблиці 2.7.

Одразу зазначимо межі цього дослід. Строгий лавинний критерій сформульовано для *бітів*: інверсія одного вхідного біта має змінювати кожен вихідний біт із імовірністю $1/2$. Тут же змінюється й підраховується символ, тому пряме порівняння отриманої частки з 50 % змінених бітів було б некоректним. Крім того, дифузія не є універсальною необхідною умовою конфіденційності: в одноразовому блокноті зміна одного біта повідомлення також змінює рівно один біт шифротексту, проте за належного використання цей шифр має досконалу секретність.

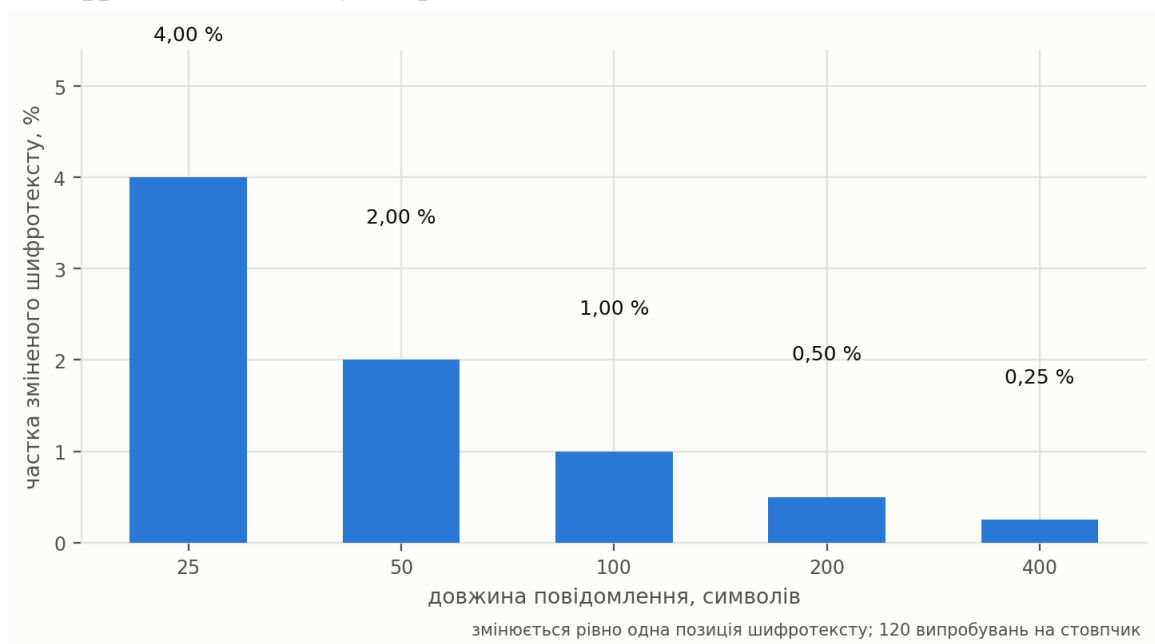


Рисунок 2.10 – Поширення зміни одного символу відкритого тексту

Таблиця 2.7 – Поширення зміни одного символу відкритого тексту

Довжина повідомлення, символів	25	50	100	200	400
Змінено символів шифротексту	1	1	1	1	1
Частка зміненого тексту, %	4,00	2,00	1,00	0,50	0,25

Результат однозначний: зміна одного символу відкритого тексту змінює **рівно один** символ шифротексту — ту саму позицію, — і так для повідомлень будь-якої довжини. Міжсимвольна дифузія відсутня: кожна позиція шифрується незалежно від решти. Практичний наслідок для криптоаналізу полягає в тому, що часткове відновлення ефективного ключа одразу робить читабельними відповідні позиції повідомлення. Слабкість же цієї конструкції обґрунтовується не самою відсутністю дифузії, а показаними вище атаками на ефективний ключ.

Порівняння зі справжньою машиною «Енігма»

Під час роботи виявлено цікавий факт: набір роторів з умови задачі — це справжні проводки роторів машини Enigma I (Wehrmacht, 1930 рік), але перелічені у зворотному порядку. Ротору, названому в умові «**ROTOR I**», відповідає історичний ротор III, «**ROTOR II**» — історичний ротор II, а «**ROTOR III**» — історичний ротор I.

Попри це, розглянута модель не є машиною «Енігма», а лише її суттєвим спрощенням. Основні відмінності зведено в таблиці 2.8.

Таблиця 2.8 – Порівняння справжньої Enigma I та моделі із задачі

Властивість	Справжня Enigma I	Модель із задачі
Обертання роторів	так, після кожного символу	ні, ротори нерухомі
Період шифру	$26 \cdot 25 \cdot 26 = 16\,900$	26
Рефлектор	так (UKW-B/C)	немає
Комутаційна панель	так, 10 пар кабелів	немає
Вибір і порядок роторів	3 з 5, тобто 60 варіантів	задається на вході
Початкові положення	$26^3 = 17\,576$	один зсув, 26
Самошифрування літери	неможливе (вада рефлектора)	можливе

Варто підкреслити окремо: за «сирим» розміром простору конфігурацій спрощена модель виглядає навіть більшою за історичну машину. Проте історичний криптоаналіз спирався на структурні властивості машини, відомі фрагменти повідомлень і недоліки експлуатації, а цю модель ослаблює відсутність крокування роторів: період алфавітів падає до 26, а пошук ключа зводиться до однієї підстановки. Отже, розмір простору конфігурацій сам собою не визначає стійкості.

Висновки

1. Реалізовано шифр, що поєднує інкрементний зсув Цезаря і каскад із трьох підстановок. Розв'язок надіслано на CodinGame: пройдено шість видимих тестів, оцінка прихованих валідаторів — 100 %. Реалізацію продубльовано двома мовами (Python і C#) та покрито 86 автоматичними тестами.

2. Алгоритм оптимізовано структурно: композиція роторів є однією підстановкою, тож її обчислюють один раз до обробки повідомлення. Попередні таблиці зменшують кількість операцій на символ і не залежать за розміром від довжини повідомлення.

3. Головний результат стосується саме дослідженої конструкції: каскад нерухомих роторів згортається в одну підстановку, а початковий зсув поглинається ефективним ключем D. Тому номінально різні конфігурації задають тотожне перетворення, а кількість ефективних ключів дорівнює $26!$. Узагальнювати це на довільні примітиви підстав немає.

4. Кількісно: ентропія номінального простору конфігурацій $\approx 269,8$ біта, ефективного ключа — $88,4$ біта, а за фіксованих відомих роторів — лише $4,7$ біта. Кожному ефективному ключу відповідає $(26!)^2 \cdot 26 \approx 2^{181,5}$ конфігурацій, існування яких підтверджено колізією.

5. Шифр піддався всім чотирьом розглянутим сценаріям атак: найдешевший із них відновлює повний ключ за один запит із 26 символів. Атака лише за шифротекстом дала повне відновлення в усіх 12 випробуваннях для довжин 160 і 240 символів; для коротших частка успіхів була меншою.

6. Основна вада — нерухомі ротори. Саме крокування давало історичній «Енігмі» період 16 900; без нього послідовність алфавітів повторюється через 26 позицій. Шифр лишається поліалфавітним, але пошук ключа зводиться до однієї підстановки D з урахуванням відомого позиційного зсуву.

7. Практичний урок: висока емпірична ентропія окремих символів ($4,689$ з $4,700$ біта) і майже рівномірний частотний розподіл не є доказом стійкості — вони не виключають позиційних залежностей.

Список використаних джерел

1. CodinGame. Encryption/Decryption of Enigma Machine [Електронний ресурс]. – Режим доступу: <https://www.codingame.com/training/easy/encryption-decryption-of-enigma-machine>
2. Caesar cipher [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Caesar_cipher
3. Enigma machine [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Enigma_machine
4. Enigma rotor details [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Enigma_rotor_details
5. Index of coincidence [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Index_of_coincidence
6. Shannon C. E. Communication Theory of Secrecy Systems // Bell System Technical Journal. – 1949. – Vol. 28, No. 4. – P. 656–715.
7. Beker H., Piper F. Cipher Systems: The Protection of Communications. – London: Northwood Books, 1982. – 436 p.
8. Singh S. The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography. – New York: Anchor Books, 2000. – 411 p.
9. Вихідний код роботи [Електронний ресурс]. – Режим доступу: <https://github.com/RE22GDV/LR2-Enigma>

Додаток А

Розв'язок задачі, надісланий на платформу CodinGame

Нижче наведено повний текст файла `solution/codingame_solution.py` — саме цей код надіслано на платформу CodinGame й оцінено валідаторами на 100 %. Решта вихідних текстів роботи (ядро шифру, модуль криптоаналізу, статистичні метрики, набір із 86 автоматичних тестів, скрипти експериментів і незалежна реалізація мовою C#) не наводиться тут, щоб не переобтяжувати звіт, і доступна в репозиторії: <https://github.com/RE22GDV/LR2-Enigma>

```
"""
CodinGame - "Encryption/Decryption of Enigma Machine" (Easy)
https://www.codingame.com/training/easy/encryptiondecryption-of-enigma-machine

Self-contained solution, submitted as-is (validator score: 100%).
This file is kept ASCII-only and byte-identical to the accepted submission;
the Ukrainian write-up lives in README.md and in src/enigma/.

Input format:
  line 1   : ENCODE | DECODE
  line 2   : starting shift N
  lines 3-5: three rotors (26 uppercase letters each)
  line 6   : the message

Encryption:
  c[i] = S[ (m[i] + N + i) mod 26 ],    S = R3 . R2 . R1
Decryption (exact inverse):
  m[i] = ( S^-1[c[i]] - N - i ) mod 26

The three rotors are composed into a single substitution S *before* the
message loop, because a composition of bijections is again a bijection.
That leaves exactly one table lookup per character:
time O(L + 26k), extra memory O(26).
"""

import sys

A = ord("A")
M = 26

def main() -> None:
    data = sys.stdin.read().split()
    mode = data[0].upper()
    shift = int(data[1]) % M
    rotors = data[2:5]
    message = data[5] if len(data) > 5 else ""

    # Collapse the rotor cascade into one substitution table.
    s = list(range(M))
    for wiring in rotors:
        table = [ord(ch) - A for ch in wiring]
        s = [table[x] for x in s]

    if mode == "ENCODE":
        out = [
```

```

        chr(A + s[(ord(ch) - A + shift + i) % M])
        for i, ch in enumerate(message)
    ]
else:
    inv = [0] * M
    for i, v in enumerate(s):
        inv[v] = i
    out = [
        chr(A + (inv[ord(ch) - A] - shift - i) % M)
        for i, ch in enumerate(message)
    ]

print("".join(out))

if __name__ == "__main__":
    main()

```